Welcome back to the masterclass! Aapka LangChain framework ka understanding ab kaafi mature ho raha hai.
Agar aap Spring Boot mein layered architecture banate waqt **Data Transfer Objects (DTOs)** ka use karte hain taaki frontend aur database ke beech data ka structure strict aur predictable rahe, toh LangChain mein **Output Parsers** aapko bilkul familiar lagenge. LLMs natural language generate karte hain, par hamare backend systems ko strict objects, JSON, ya arrays chahiye hote hain. Yahi gap Output Parsers fill karte hain.
Let's master Output Parsers step-by-step.

Module 1: The Need & Foundation (Topics 1, 2, 3, 4)

1. Why raw LLM output is difficult to use & 2. What are Output Parsers?

Raw LLM APIs hamesha ek unstructured text (string) ya AIMessage object return karti hain. Agar aapko LLM se top 5 array sorting algorithms ki list chahiye taaki aap unhe UI drop-down mein dikha sakein, aur LLM reply kare: *"Sure! Here are the top 5 algorithms: 1. QuickSort..."* toh aapka code crash ho jayega kyunki usey strictly ["QuickSort", "MergeSort", ...] expected tha.
Output Parsers LangChain ke wo components hain jo:

1. LLM ko strictly batate hain ki output kis format mein dena hai (Format Instructions).
2. LLM ke text response ko explicitly list, JSON, ya custom Python objects mein convert karte hain.

3. Different types of Output Parsers

Industry mein mainly 3 parsers use hote hain:

- StrOutputParser: Clean string ke liye (Humne pichli class mein dekha tha).
- JsonOutputParser: Dynamic JSON objects return karne ke liye.
- PydanticOutputParser: Strict type-checking aur validation ke sath custom objects banane ke liye (Industry Standard).

4. StrOutputParser (Quick Recap)

- **Internal Working:** Yeh simply AIMessage.content ko extract karta hai.
- **Data Flow:** AIMessage(content="Hello", metadata={...}) -> StrOutputParser -> "Hello"
- **Use Case:** Chatbots jahan sirf text dikhana hai.

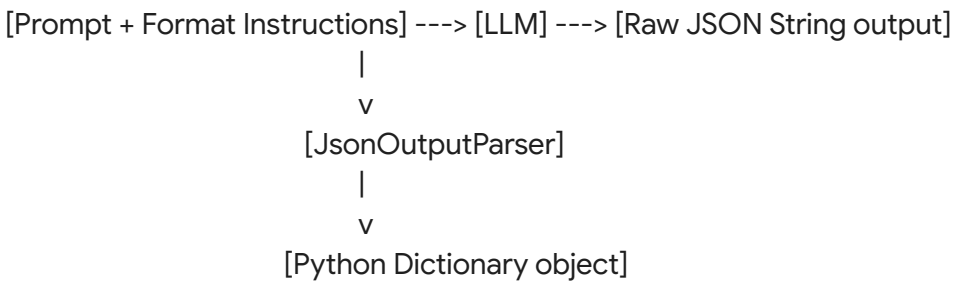
Module 2: Structured Data Extraction (Topics 5 & 7)

5. JsonOutputParser & 7. Creating structured JSON responses

Jab hume LLM se multiple fields extract karne hote hain (e.g., entity extraction), hum JsonOutputParser use karte hain. Yeh ensure karta hai ki output ek valid Python dictionary ban kar aaye.

- **Real-world Analogy:** Ek data entry operator jo ek raw paragraph padhta hai aur usme se Name, Age, aur City nikal kar Excel sheet ke columns mein daal deta hai.

Data Flow Diagram:



Complete Python Code:

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import JsonOutputParser

load_dotenv()
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# 1. Initialize the Parser
parser = JsonOutputParser()

# 2. Create the Prompt (CRITICAL: Inject format instructions)
template = """
Extract the user details from the following text.
Text: {text}

{format_instructions}
"""
prompt = PromptTemplate(
    template=template,
    input_variables=["text"],
```

```
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

# 3. Build Chain
chain = prompt | llm | parser

# 4. Execute
text_input = "Gagan is a 24 year old software engineer living in Bengaluru."
result = chain.invoke({"text": text_input})

print(type(result)) # Output: <class 'dict'>
print(result)
```

Dry Run:
Line 15 mein humne parser.get_format_instructions() ko as a partial variable prompt mein inject kiya. Isne implicitly LLM ko ek hidden prompt bheja: *"Please output valid JSON only"*. Jab LLM ne {"name": "Gagan", "age": 24, ...} as string return kiya, parser ne usey properly Python dict mein parse kar diya.
Common Mistakes:

- Prompt mein {format_instructions} inject karna bhool jana. Iske bina LLM plain text dega aur JSON parser crash ho jayega.

Module 3: Advanced Validation & Schemas (Topics 6, 8, 9)

6. PydanticOutputParser & 8. Creating schemas

JSON theek hai, par wo strict nahi hai. Agar hume strictly ensure karna hai ki age integer hi ho, string na ho, toh hum **Pydantic** ka use karte hain. Pydantic Python ka standard validation library hai (just like Bean Validation in Spring Boot).

- **Real-world Analogy:** Airport security check. JSON Parser sirf ye dekhta hai ki aapke paas bag (JSON format) hai ya nahi. Pydantic Parser bag khol kar check karta hai ki andar items strictly rule-book (Schema) ke according hain ya nahi.

9. Validation of LLM responses (Code Execution & Dry Run)

```
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser
```

```
# 1. Define the Schema using Pydantic
class DSAConcept(BaseModel):
    concept_name: str = Field(description="Name of the data structure or algorithm")
    time_complexity: str = Field(description="Big O time complexity")
    is_linear: bool = Field(description="True if it is a linear data structure")

# 2. Initialize Parser with Schema
pydantic_parser = PydanticOutputParser(pydantic_object=DSAConcept)

# 3. Create Prompt
prompt = PromptTemplate(
    template="Explain this tech concept briefly.\nConcept: {query}\n\n{format_instructions}",
    input_variables=["query"],
    partial_variables={"format_instructions": pydantic_parser.get_format_instructions()}
)

# 4. Build and Invoke Chain
chain = prompt | llm | pydantic_parser

result = chain.invoke({"query": "Explain Queues"})
print(type(result)) # Output: <class '__main__.DSAConcept'>
print(f"Name: {result.concept_name}, Linear: {result.is_linear}")
```

Expected Output: Name: Queue, Linear: True
Internal Working:
Yahan Field(description="...") bohot important hai. Pydantic parser in descriptions ko read karke automatically ek bohot detailed prompt banata hai jo LLM ko strictly batata hai ki kaunsa field kaisa hona chahiye. Output ek raw dictionary nahi, balki ek strict Python Class object hota hai.

Module 4: Error Handling & Retry Strategies (Topic 10)

10. Error handling and retry strategies

Kayi baar LLM bad formatting de deta hai (e.g., JSON ke end mein ek comma miss kar dena). Aise mein system crash ho jata hai (OutputParserException). Isko handle karne ke liye hum OutputFixingParser ya RetryOutputParser use karte hain.

- **Internal Working:** Agar initial parser fail hota hai, OutputFixingParser us galat output ko pakadta hai, aur ek naye LLM call mein raw output aur schema bhej kar kehta hai: *"Hey LLM, you messed up the JSON formatting here, please fix it according to this schema."*

```
from langchain.output_parsers import OutputFixingParser

# Wrap your original failing parser with a fixing parser
fixing_parser = OutputFixingParser.from_llm(parser=pydantic_parser, llm=llm)

# Now use fixing_parser in your chain instead of pydantic_parser
```

Review & Practice Segment

Interview Questions & Answers

Q: What is the primary difference between JsonOutputParser and PydanticOutputParser?
Answer: JsonOutputParser purely ensures the output is a valid JSON string and parses it into a Python dictionary. It does not enforce specific keys or data types. PydanticOutputParser enforces a strict schema, ensuring specific keys exist and data types (int, boolean, list) are strictly met, returning a typed Python object.
Q: If an LLM hallucinates and adds conversational text like "Here is your JSON: { ... }" before the actual JSON, how does the parser handle it?
Answer: Modern LangChain parsers are equipped with internal regex that scans the output and extracts everything between the first { and last } (or [and]), safely stripping away the conversational text.

Hands-on Exercises

1. **Task 1:** Create a PydanticOutputParser for a StockMarketAnalysis object that strictly expects stock_ticker (str), current_sentiment (str: bullish/bearish), and target_price (int).
2. **Task 2:** Intentionally remove {format_instructions} from your prompt template and observe the error Pydantic throws.

Mini Project: AI Medical Triage & Device Entity Extractor

Ek real-world AI ecosystem ka component banate hain. Imagine aapko raw patient queries aati hain, aur aapke backend ko structured patient symptom profiles aur required medical devices ki list chahiye triage karne ke liye.

```
from pydantic import BaseModel, Field
```

```python
from typing import List
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import PydanticOutputParser

# 1. Define Strict Schema
class MedicalTriage(BaseModel):
    patient_condition: str = Field(description="Summary of the patient's condition")
    urgency_level: str = Field(description="Must be exactly 'LOW', 'MEDIUM', or 'HIGH'")
    required_devices: List[str] = Field(description="List of medical devices or imaging tools needed (e.g., MRI, Dialysis Machine, ECG)")

# 2. Setup LLM and Parser
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
parser = PydanticOutputParser(pydantic_object=MedicalTriage)

# 3. Create Template
template = """
You are an AI diagnostic routing assistant. Analyze the clinical text below and extract the triage details.
Text: {clinical_text}

{format_instructions}
"""

prompt = PromptTemplate(
    template=template,
    input_variables=["clinical_text"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

# 4. LCEL Chain
triage_chain = prompt | llm | parser

# 5. Execute
patient_input = "Patient arrived with severe lower back pain radiating to the abdomen. Suspected renal calculi. Needs immediate scan to verify stone location."
print("Processing Medical Data...\n")
result = triage_chain.invoke({"clinical_text": patient_input})

# Output as a strict Object
print(f"Condition: {result.patient_condition}")
print(f"Urgency: {result.urgency_level}")
print(f"Devices to Ready: {', '.join(result.required_devices)}")
```

Expected Output Check: LLM will correctly identify "HIGH" urgency and list devices like "CT Scan" or "Ultrasound" purely structured as a list, completely ready to be ingested by a backend database or Neo4j graph.
Output parsing is what makes Generative AI deterministic enough for real-world software engineering. Ab jab hum prompt templates aur structured output pipelines samajh chuke hain, kya hum next class mein **Memory components** start karein taaki hamara LLM purani baaton ka context yaad rakh sake?